Tema 3.B (segunda parte): Modelado del comportamiento

Esta sección analiza y representa el comportamiento dinámico de sistemas software, cubriendo aspectos como interacciones y intercambio de mensajes con el Diagrama de Secuencia. También explora el ciclo de vida de los objetos y los cambios de estado mediante el Diagrama de Transición de Estados, así como los flujos de trabajo y procesos utilizando el Diagrama de Actividad.

Índice

3

Modelado del comportamiento

- Introducción
- Diagramas de Interacción
  - Diagrama de Secuencia
- Diagrama de Transición de Estados
- Diagrama de Actividad
- Conclusión

Modelado del Comportamiento

El modelado del comportamiento representa el **comportamiento dinámico del sistema** mientras está en funcionamiento, a diferencia de los modelos estáticos que describen su estructura. Estos modelos expresan cómo responde el software a eventos externos o estímulos del entorno durante su **ejecución** y son esenciales para comprender las interacciones, reacciones y cambios de estado que experimenta el sistema a lo largo del tiempo.

Visión Dinámica

Capturan el comportamiento del sistema durante su ejecución en tiempo real

Respuesta a Eventos

Modelan cómo el sistema reacciona ante estímulos externos y cambios en el entorno

Diagramas UML de Comportamiento

El modelado del comportamiento se representa mediante dos enfoques complementarios que capturan diferentes aspectos de la dinámica del sistema.

Diagramas Dirigidos por Eventos

Se centran en el control del sistema y en cómo este reacciona ante estímulos externos o internos. Muestran los cambios de estado o el intercambio de mensajes en momentos precisos

- Diagramas de interacción
  - Diagrama de Colaboración
  - Diagrama de Secuencia
- Diagrama de Transición de Estados

Diagramas Dirigidos por Datos

Se centran en la coordinación de procesos, mostrando el flujo de control paso a paso y, opcionalmente, cómo se pasan los datos de una actividad a otra.

- Diagrama de Actividad

Diagramas de interacción

> Estos diagramas modelan la **colaboración** y el **intercambio de mensajes** entre objetos durante la ejecución del sistema. Son fundamentales para entender cómo los componentes se comunican y coordinan para lograr funcionalidades específicas.

Diagrama de Colaboración

- Mejor para visualizar **relaciones estructurales**
- Permite agrupar visualmente los objetos según sus interacciones, no cronológicamente
- Útil para sistemas con arquitecturas complejas

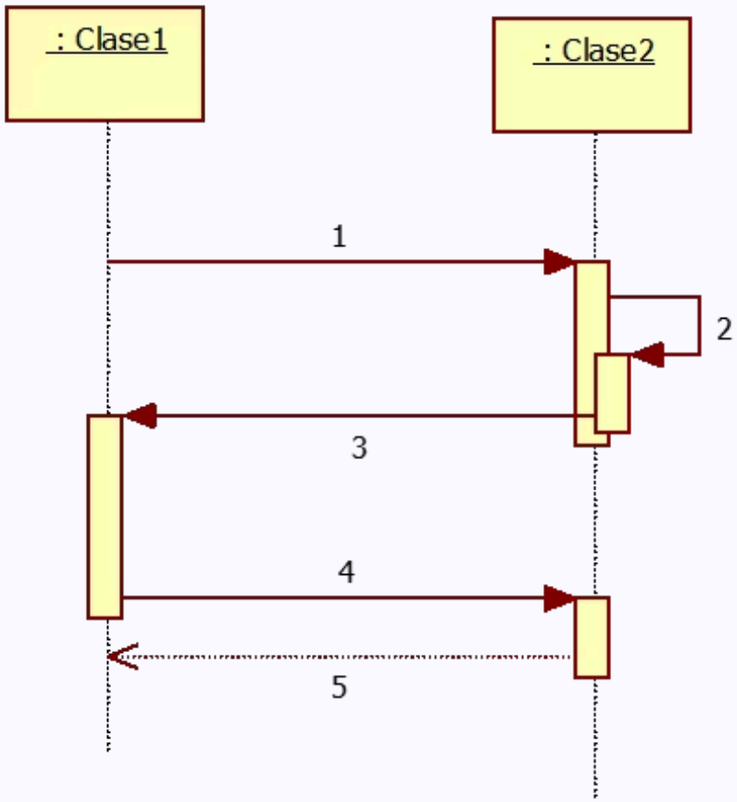
Diagrama de Secuencia

- Mejor para entender el **orden temporal**
- Permite agrupar los objetos en función de la cronología de los mensajes que intercambian
- Útil para analizar flujos de trabajo detallados

ⓘ Nos centraremos en el **Diagrama de Secuencia** ya que, al mostrar explícitamente el orden cronológico, es la herramienta más eficaz para comprender la lógica de los procesos a modelar.

Diagrama de Secuencia

Un Diagrama de Secuencia es un tipo de diagrama de interacción UML que se enfoca en la **ordenación temporal** de los mensajes. Muestra cómo los objetos y/o actores se comunican entre sí y el orden en que ocurren estos intercambios.



Orden Temporal

Resalta la secuencia de los mensajes que se intercambian los participantes.

Participantes

Incluye tanto los **actores** como los **objetos** de las clases que participan en la interacción.

Mensajes

Los mensajes se intercambian de manera ordenada a lo largo de la vida de los participantes.

En UML se representa como un gráfico bidimensional (tiempo vs participantes) que debe incluir al menos un participante (actor u objeto) y un mensaje. El participante que comienza la interacción se coloca a la izquierda.

Diagrama de Secuencia: Elementos

Actores

Entidades externas al sistema que inician o reciben interacciones. Se representan en la parte superior mediante el ícono de un actor.

Objetos

Instancias de clases participantes, representadas en la parte superior mediante un rectángulo con su nombre subrayado y precedido de ":"

Mensajes

Líneas con flechas que muestran el intercambio de información y el flujo de control.

Fragmentos

Marcos que encapsulan partes de la interacción para definir condiciones, bucles o concurrencia.

Líneas de Vida

Líneas verticales que describen la existencia y el avance temporal de cada objeto.

Focos de Control

Barras rectangulares sobre la línea de vida que indican cuándo el objeto está activo ejecutando una operación.

Si un elemento es simultáneamente actor y objeto se presenta con el ícono de actor y precediendo su nombre con ":" (p.e., ": Persona" sería un actor externo que también representa un objeto Persona dentro del sistema)

Diagramas de Secuencia: Tipos de Mensajes

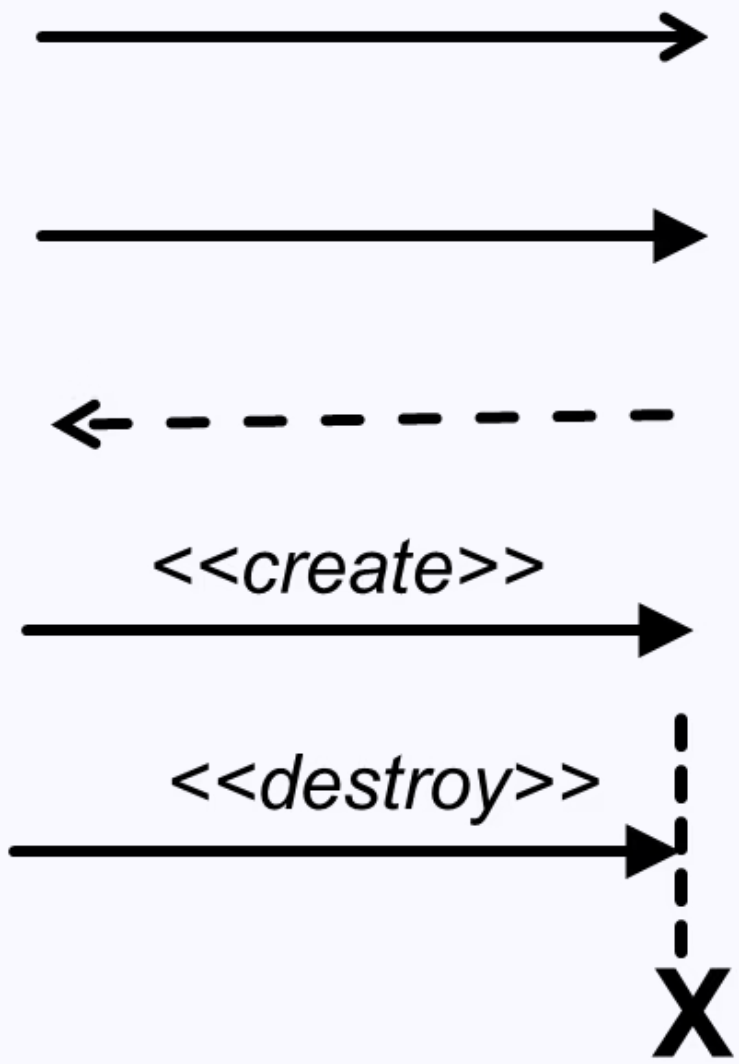
Mensaje Asíncrono: El objeto emisor envía el mensaje y continúa su ejecución sin esperar una respuesta.

Mensaje Síncrono: El objeto emisor envía el mensaje y pausa su ejecución hasta recibir una respuesta.

Notificación de Respuesta: Indica la contestación o el valor de retorno de un mensaje síncrono previo.

Creación de Objetos: Un mensaje que representa la instanciación de un nuevo objeto.

Destrucción de Objetos: Un mensaje que señala el fin de la existencia de un objeto.



Diagramas de Secuencia: Normas de creación

1

Participantes

- Todos los participantes, excepto los actores externos, deben ser objetos que se corresponden con una clase del sistema. Nombrarlos usando `nombre:Clase` o `:Clase`.
- El iniciador (normalmente un actor) se coloca a la izquierda. Como norma, objetos de izquierda a derecha según interacción.
- La línea de vida de un objeto que no existe desde el inicio debe empezar en el punto vertical exacto donde recibe el mensaje de creación (`<<create>>`).

2

Mensajes

- Los mensajes se representan horizontalmente.
- Todo mensaje enviado debe corresponder con un método público en la clase del objeto receptor (salvo mensajes de retorno).
- Un objeto puede enviarse un mensaje a sí mismo generando un foco de control apilado.
- Distinguir rigurosamente los tipos de mensaje:
  - Llamada síncrona: Emisor espera respuesta.
  - Llamada asíncrona: Emisor no espera.
  - Respuesta: Devuelve control y/o dato.
  - Creación/Destrucción: Crea o destruye objeto.

3

Temporalidad y Foco de Control

- El tiempo transcurre verticalmente de arriba a abajo; no se permiten flechas ascendentes.
- El foco de control (rectángulo de activación) es obligatorio mientras el objeto ejecuta una tarea o espera una respuesta.

4

Lógica Estructural

- No usar flechas cruzadas para bucles o saltos.
- Usar fragmentos para condiciones y bucles:
  - `alt`: decisiones excluyentes (if...else).
  - `opt`: decisiones simples (if sin else).
  - `loop`: iteraciones (for/while).

Diagrama de Secuencia: Ejemplo

El caso de uso a modelar es **'Servir Café'**. Tras la selección y pago del café, se evalúa si procede devolvución de cambio. El objeto `:Maquina` asume el rol de controlador, enviando mensajes síncronos a los subsistemas (Dispensadores de Vaso y Café) para orquestar la preparación física. El flujo concluye exitosamente cuando el sistema recibe las confirmaciones internas y notifica al Cliente para la recogida final.

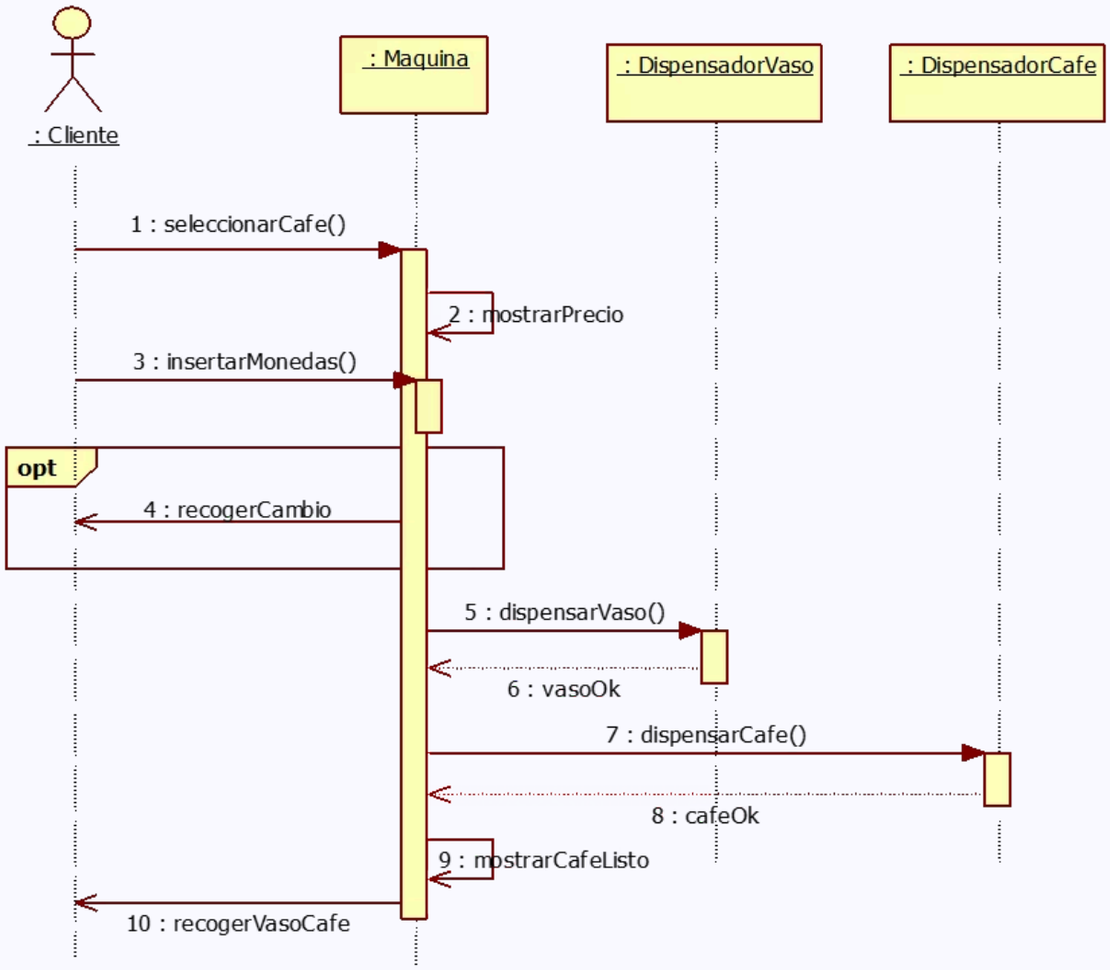
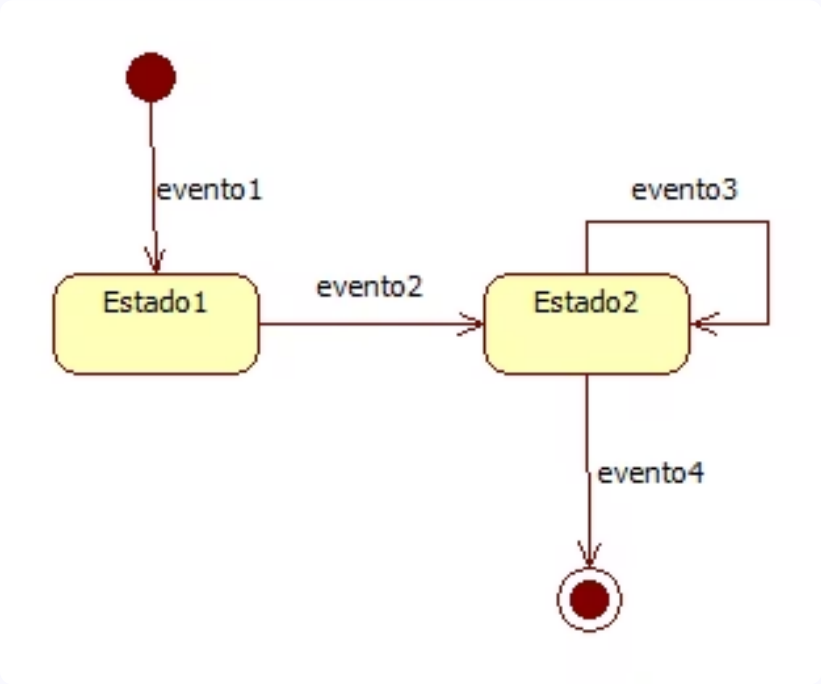


Diagrama de Transición de Estados (DTE)

El DTE representa los diferentes **estados** por los que puede pasar un objeto/sistema y las **transiciones** entre ellos provocadas por eventos específicos. El DTE captura cómo un objeto/sistema mantiene información sobre su estado actual y **responde de manera diferente a los mismos eventos según su estado**. Es fundamental para modelar el ciclo de vida de objetos con comportamiento dependiente del estado.



En UML se representa como un grafo dirigido que requiere obligatoriamente definir el estado inicial, una transición y al menos un estado destino.

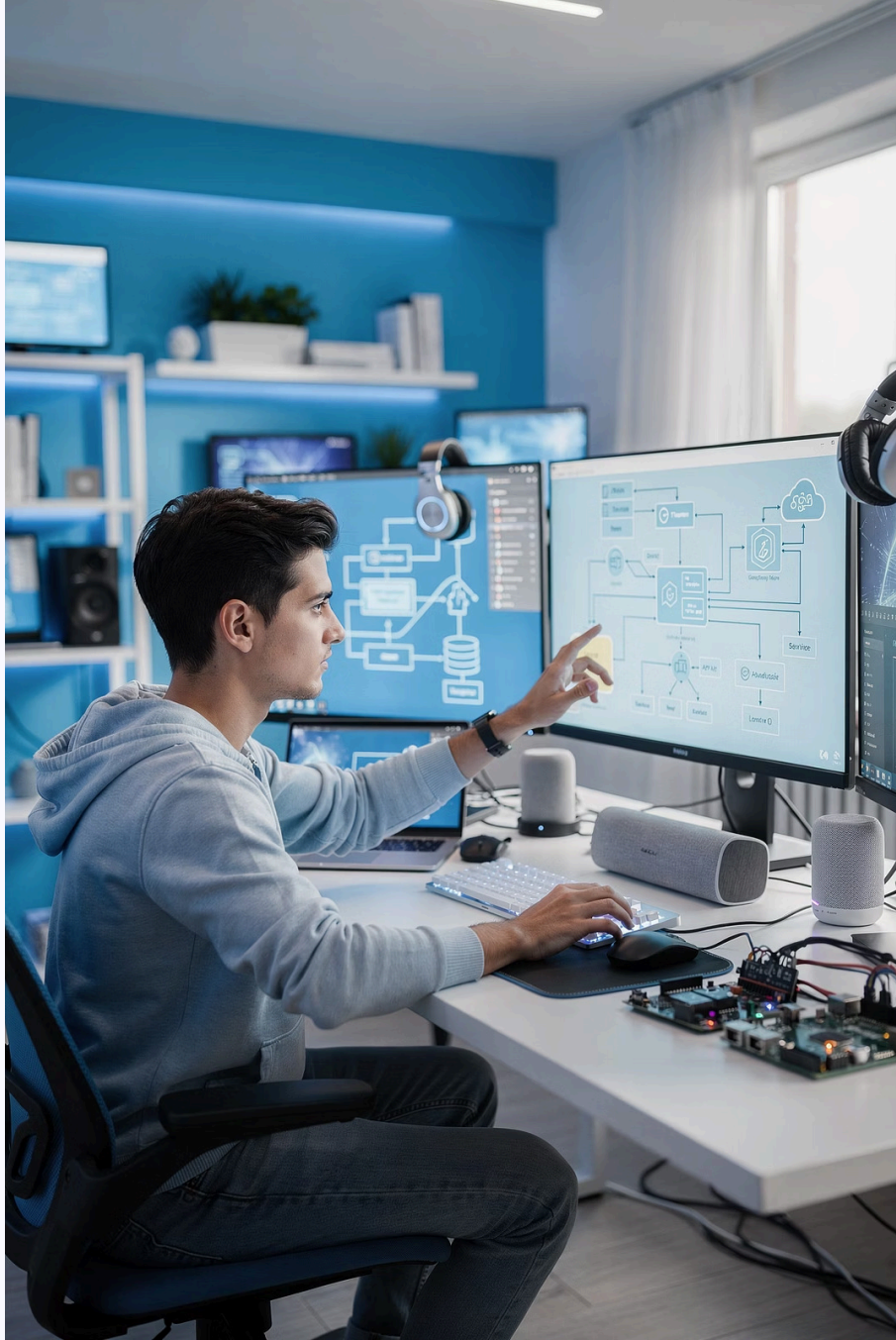
DTE: Perspectivas de modelado

Estado del Sistema

Representa cómo se percibe el sistema desde fuera, basándose en la función que está realizando en un momento dado. Este punto de vista se centra en la experiencia del usuario o en la interacción global con otros sistemas.

Estado de cada Objeto

Describe el estado individual de los objetos que componen el sistema mientras este ejecuta su función. Se enfoca en cómo los atributos de una instancia de clase cambian en respuesta a eventos, afectando su comportamiento.



DTE: Elementos

Estados

Condiciones o situaciones en la vida de un objeto/sistema durante las cuales satisface alguna condición y tiene determinado comportamiento.

Transiciones

Cambios de un estado a otro, desencadenados por eventos o condiciones específicas

Eventos

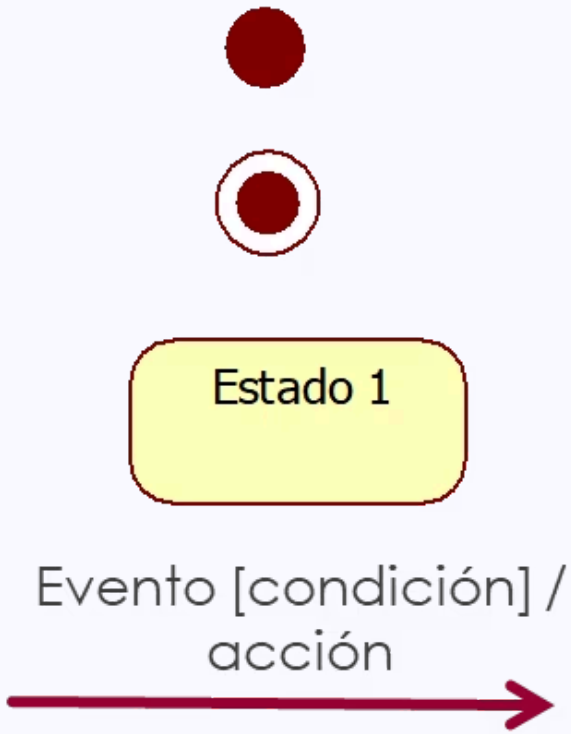
Estímulos que provocan las transiciones, pueden ser señales, llamadas, cambios de condición o paso del tiempo.

Acciones

Operaciones que se ejecutan como respuesta a un evento durante una transición

DTE: Sintaxis en UML

Los Diagramas de Transición de Estados en UML siguen una sintaxis específica para representar de manera clara el comportamiento dinámico de un objeto o sistema.



Estado Inicial

Estado Final

Estado

Transición

DTE: Normas de creación

1

Ciclo de Vida

- **Un solo estado inicial:** Siempre se describe y es único.
- **N estados finales:** Puede haber varios estados finales
- **Regla de Destrucción:** En los DTE (de objetos), al menos un estado final debe ser alcanzable por una transición.
- **Sistemas Infinitos:** En los DTE de sistemas es posible no tener estado final si se representa un sistema perpetuo (p.e., satelite).

2

Transiciones

- **Definición:** `Evento [Condición] / Acción`.
- **Transiciones Reflexivas:** Un evento puede no producir un cambio de estado.
- **Principio de Determinismo:** No pueden existir dos transiciones con el mismo evento desde un mismo estado sin condiciones mutuamente excluyentes.

3

Topología y Coherencia

- **Evitar "Agujeros Negros":** Todo estado (salvo el final) debe tener transición de salida.
- **Evitar "Milagros":** Todo estado (salvo el inicial) debe tener transición de entrada.
- **Consistencia con clases:** Los eventos de transición deben corresponder a operaciones públicas de la clase modelada o señales externas definidas.

Diagrama de Transición de Estados: Ejemplo

Una clase **Persona** que, está presente en un diagrama de clases, quiere representar el estado laboral de una persona. El ciclo de la persona comienza como 'Menor' hasta los 18 años, entrando en una etapa activa donde puede alternar cíclicamente entre el desempleo (pudiendo estudiar) y el trabajo. Esta etapa se interrumpe al cumplir 65 años, forzando la transición a 'Jubilado' desde cualquier estado activo. Finalmente, el objeto termina su ciclo con el evento 'morir'.

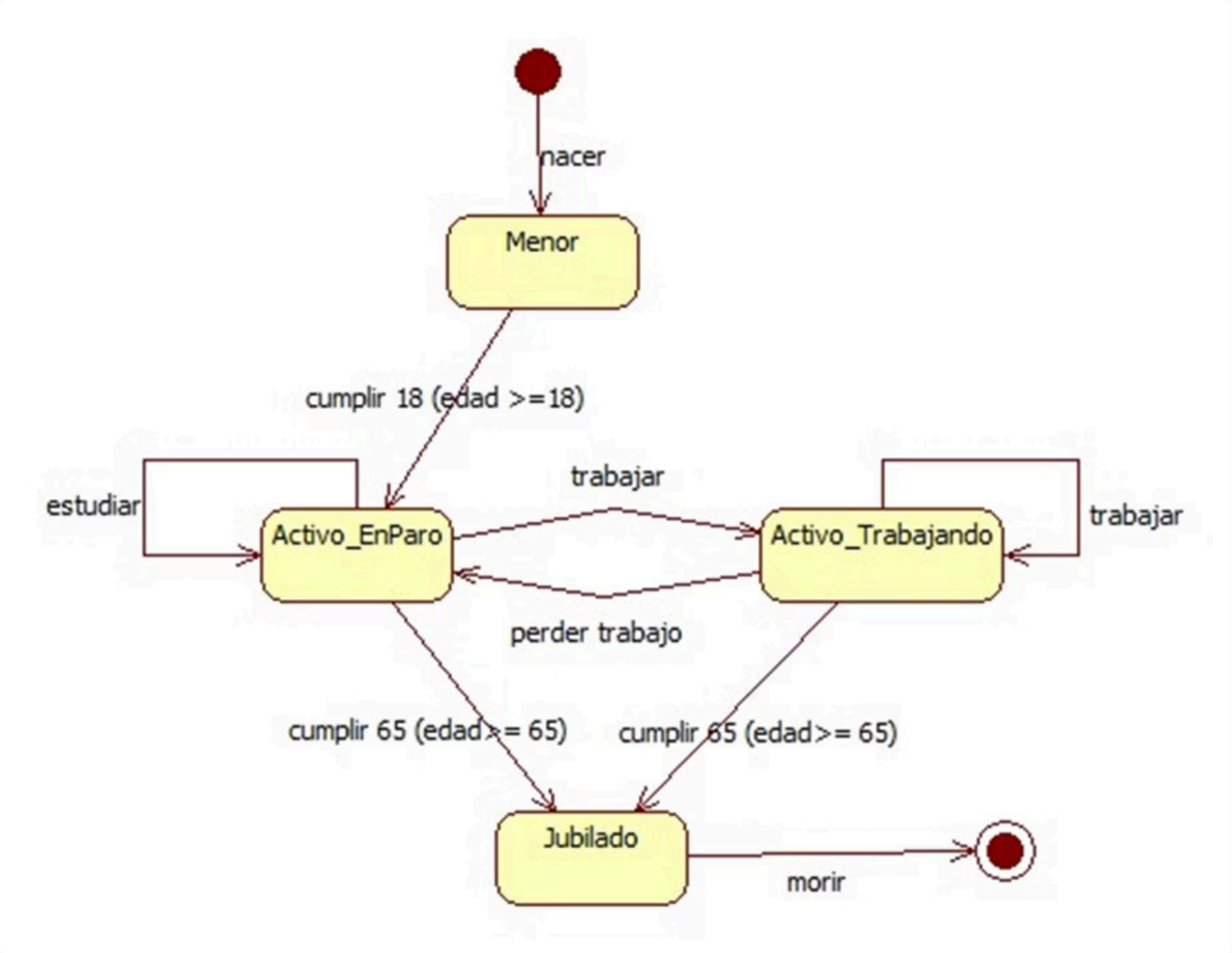
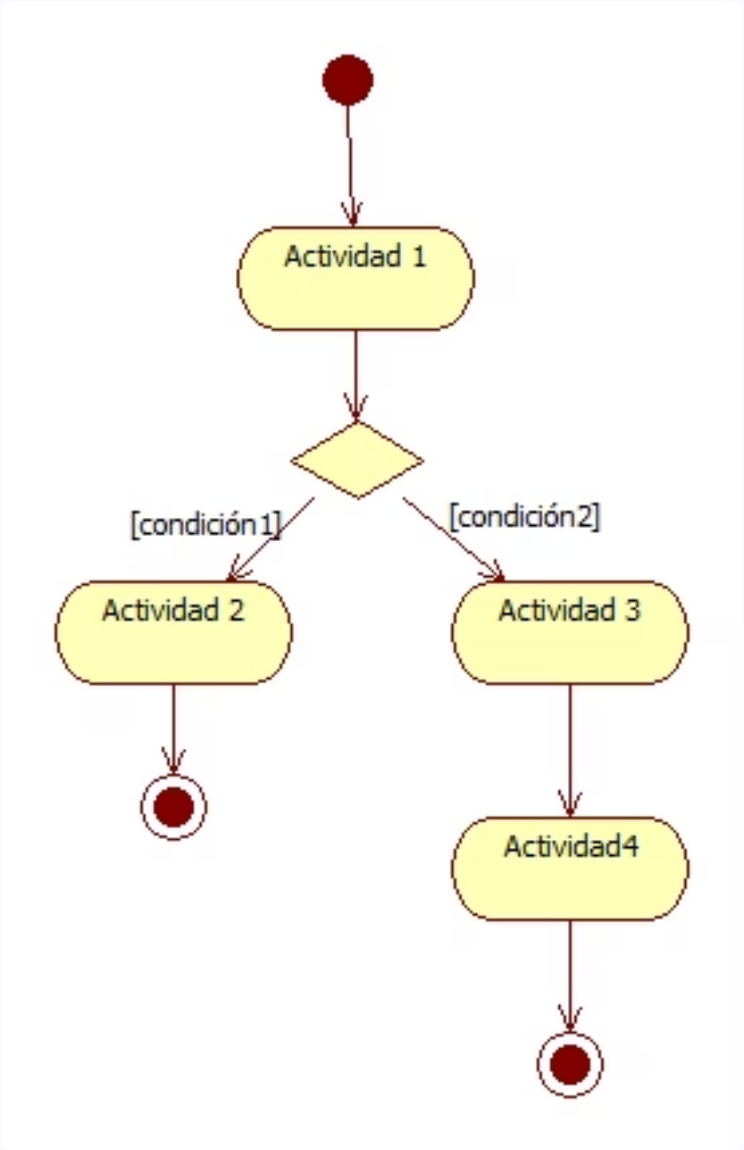


Diagrama de Actividad

El Diagrama de Actividad es un tipo de diagrama **dirigido por eventos** que modela el **flujo de trabajo** y las **actividades** dentro de un proceso. Representa actividades que significan la ejecución de un método o el desarrollo de una tarea específica.



Enfoque en Procesos

A diferencia del DTE (que se centra en estados de objetos o sistemas), el diagrama de actividad muestra el flujo general de control y datos a través de actividades concurrentes o secuenciales.

Aplicaciones Prácticas

Es especialmente útil para modelar la lógica interna de un método complejo o el flujo de eventos de un Caso de Uso donde intervienen múltiples actores.

En UML, se representa como una sucesión de acciones conectadas cronológicamente mediante flechas (flujos), que pueden bifurcarse usando nodos de decisión y paralelizarse usando líneas de sincronía.

Elementos del Diagrama de Actividad

Nodos Iniciales y Finales

Indican dónde comienza y termina el flujo de actividades

Actividades

Representan las tareas o acciones que se ejecutan en el proceso

Flujos de Control

Flechas que conectan actividades mostrando la secuencia de ejecución

Decisiones y Bifurcaciones

Nodos que permiten ramificar el flujo según condiciones o en paralelo

Sincronización

Nodos que coordinan flujos paralelos de actividades

Carriles (*Swimlanes*)

Dividen el diagrama por responsables o sistemas que ejecutan las actividades

Diagrama de actividad: Sintaxis en UML

Los Diagramas de Actividad en UML siguen una sintaxis específica para representar de manera clara el flujo de control y la secuencia de acciones dentro de un proceso o caso de uso.

Diagramas de Actividad: Normas de creación

1

Estructura y Delimitación

- **Punto de Partida:** Debe existir un único nodo inicial que marca el comienzo del proceso.
- **Finalización del Proceso:** Por lo general hay uno o más nodos finales que detienen todo el proceso.
- **Direccionalidad:** El flujo debe dibujarse preferentemente de arriba a abajo o de izquierda a derecha, facilitando la lectura temporal.

2

Acciones y Flujo de Control

- **Sintaxis de Actividad:** Se recomienda nombrar las actividades con la fórmula "Verbo + Sustantivo" (ej. Validar Usuario).
- **Nodo de decisión:** De un nodo de decisión salen dos o más flujos, cada uno con una condición de guarda entre corchetes. Estas condiciones deben ser mutuamente excluyentes y cubrir todas las posibilidades.

3

Concurrencia

- **Barras de Sincronización:** Se usan barras gruesas para gestionar el paralelismo.
- **Ramificación:** Una entrada, múltiples salidas. Genera hilos paralelos que se ejecutan simultáneamente.
- **Unión:** Múltiples entradas, una salida. Es bloqueante: el proceso no continúa hasta que todos los flujos de entrada hayan llegado a la barra.

Diagrama de actividad: Ejemplo

El sistema a modelar controla una **máquina expendedora automática**. Tras recibir la solicitud y validar existencias, el flujo se divide: el zumo se dispensa de inmediato, pero el café inicia un proceso complejo y concurrente. Para optimizar el tiempo, la máquina realiza dos tareas en paralelo: gestionar la preparación física (carga de filtro, agua y calentado) y disponer la vajilla y complementos (taza, cuchara y azúcar opcional). Finalmente, el sistema sincroniza ambos hilos, asegurando que el café solo se vierta cuando tanto la bebida como la taza estén listas.

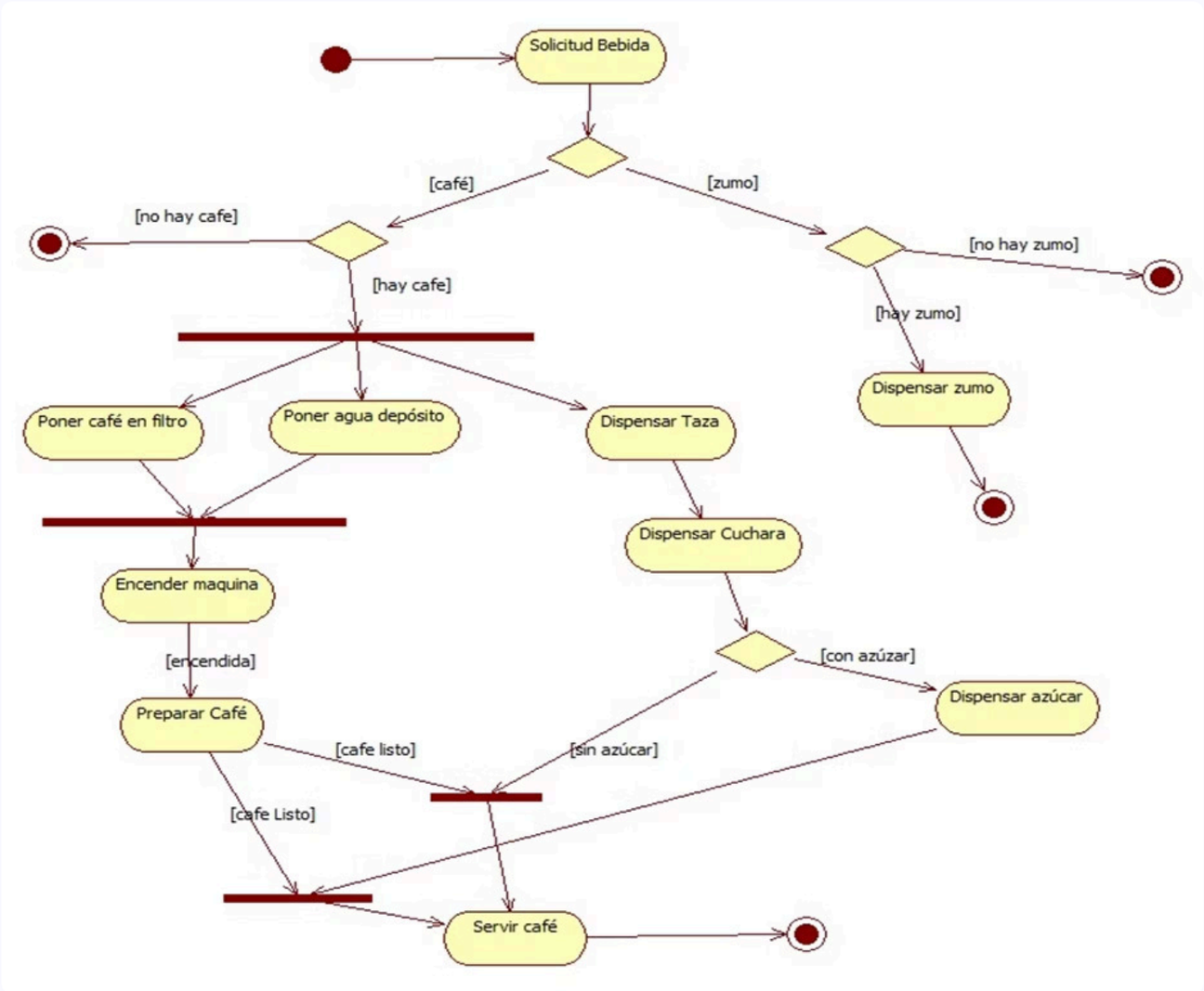
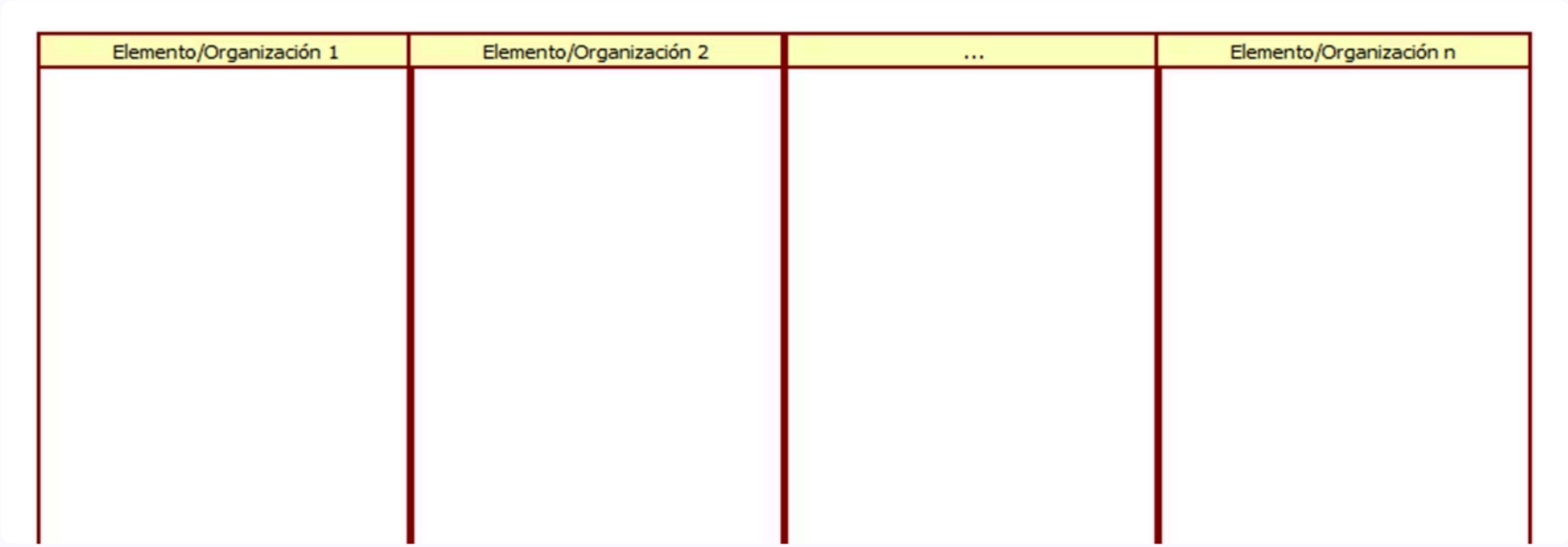


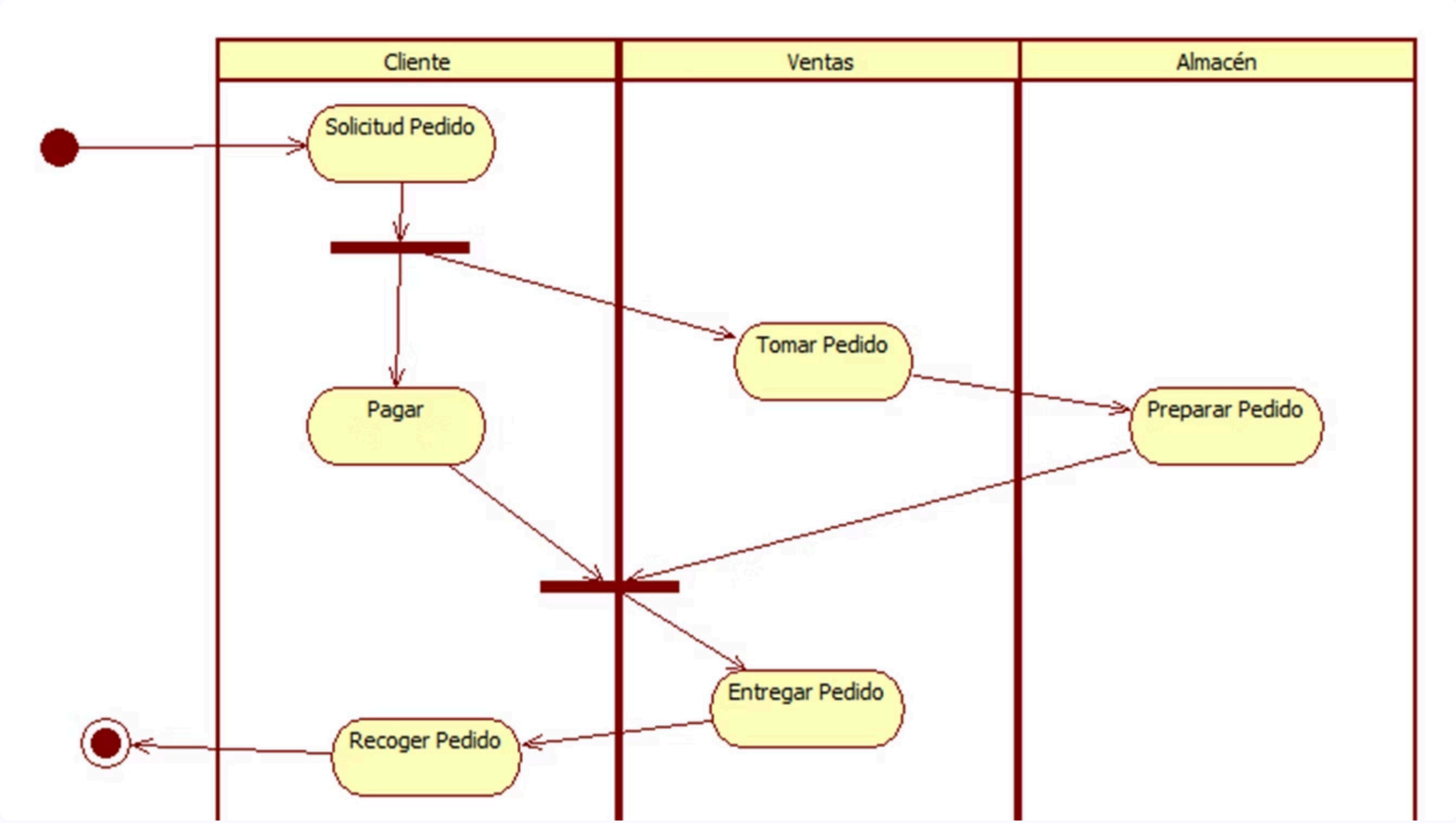
Diagrama de actividad con *swimlanes*

Los swimlanes, o carriles de natación, son un **mecanismo de organización visual** que se puede emplear en los diagramas de actividad que divide el mismo en zonas de responsabilidad exclusiva. Cada calle representa una clase, componente, actor o departamento, permitiendo visualizar no solo qué ocurre, sino quién ejecuta cada actividad y cómo interactúan las distintas partes del sistema entre sí.



Ejemplo diagrama de actividad (*swimlanes*)

El proceso a modelar se inicia cuando el Cliente solicita un pedido, activando simultáneamente el trámite del pago y la preparación en Almacén (previa gestión de Ventas). Cuando ambas tareas han finalizado (pedido cobrado y preparado), Ventas entrega el producto para que el Cliente lo recoja.



Resumen: ¿Cuándo usar cada diagrama de comportamiento?

Diagrama de Secuencia

- Necesites modelar la interacción entre objetos
- Quieras mostrar el orden temporal de los mensajes
- Analices escenarios específicos de casos de uso
- Necesites documentar la comunicación entre componentes

Diagrama de Transición de Estados

- Necesites modelar el ciclo de vida de un objeto o sistema
- El comportamiento de un objeto o sistema dependa de su estado
- Existen eventos externos que cambian el estado del objeto

Diagrama de Actividad

- Necesites modelar un flujo de trabajo completo
- Analices procesos de negocio (casos de uso)o algoritmos (métodos) complejos
- Haya múltiples actores o sistemas involucrados
- Existan actividades paralelas o concurrentes

Estos diagramas no son opciones alternativas, sino complementarios. Normalmente se usan simultáneamente para definir las tres dimensiones del comportamiento del software: el flujo (Actividad), la interacción (Secuencia) y el ciclo de vida (Estados).